Sort K Sorted Array

AlgoExpert · Hard · Heaps

The problem

- Every element is at most k positions from its sorted position; sort with a size-(k+1) min-heap.

Intuition

- Every element is within k of its spot, so the minimum of the next k plus one items is always next in order.

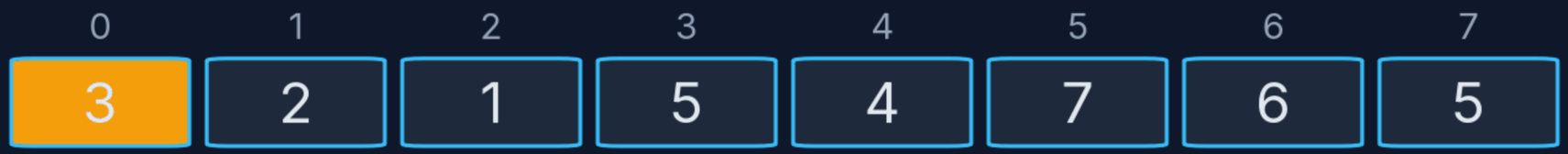
Brute force vs optimal

- Full sort → O(n log n)
- Insertion sort → O(n·k)
- Min-heap of size k + 1 → O(n log k)

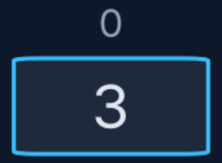
Approach

- 1. Fill a min-heap with k plus one items
- 2. Pop the min into place
- 3. Push the next item
- 4. Drain the heap

Worked example



heap (size ≤ 4)



sorted output

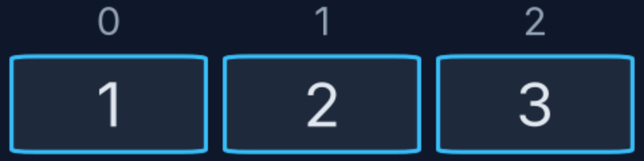


Add 3

Worked example



heap (size ≤ 4)



sorted output



Add 1

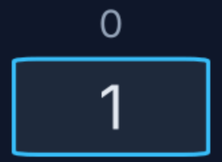
Worked example



heap (size ≤ 4)



sorted output



Add 5, pop 1

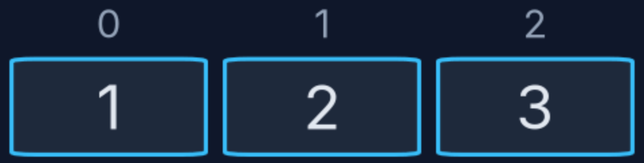
Worked example



heap (size ≤ 4)



sorted output



Add 7, pop 3

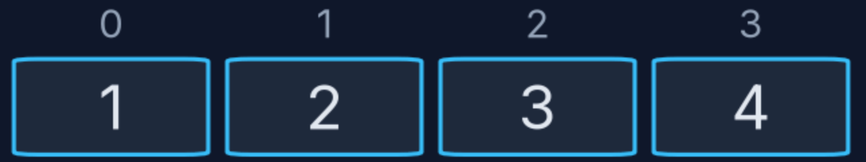
Worked example



heap (size ≤ 4)

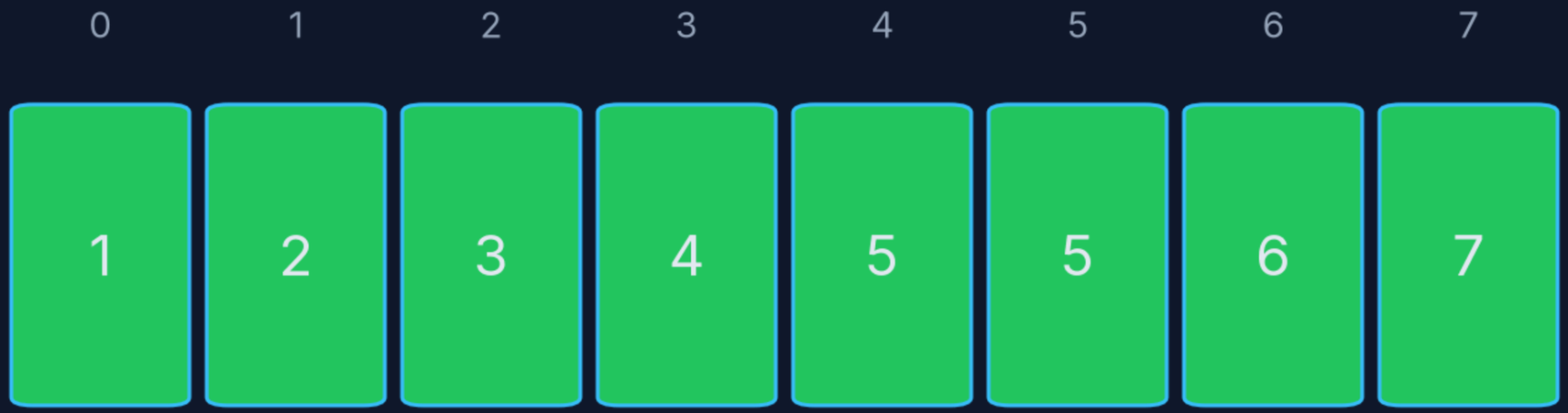


sorted output



Add 6, pop 4

Worked example



Drain the heap → sorted

Java solution

```java
public class SortKSortedArray {
    public static int[] sortKSortedArray(int[] a, int k) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        int w = 0;
        for (int i = 0; i < a.length; i++) {
            pq.add(a[i]);
            if (pq.size() > k) a[w++] = pq.poll();
        }
        while (!pq.isEmpty()) a[w++] = pq.poll();
        return a;
    }
}
```

Complexity

- Time: O(n log k)
- Space: O(k)

Edge cases & pitfalls

- K = 0 → already sorted
- K ≥ n
- Write back in place

Interview tips & follow-ups

- Explain why the min of k+1 is safe

Recap

- Every element is within k of its spot, so the minimum of the next k plus one items is always next in order.
- Complexity: O(n log k)